ICTlevels design document

Oct 18, 2026 · @Francisco

Overview

ICTlevels is a Pine Script v6 overlay that draws time-anchored ICT reference levels on NQ (New York time) and DAX (Madrid time) charts. It computes no signals, raises no alerts and runs no backtest: it plots levels and leaves their reading to the trader.

Every object is placed with `xloc.bar_time`, so lines and labels survive timeframe changes and are re-anchored to the right margin on each update. All logic is gated by `tfok` (chart timeframe of 1 hour or lower) and, per module, by narrower limits shown below.

Module	What it draws	Chart timeframe
Days and weeks	Day vertical lines, running day high and low, equilibrium (eq), midnight open (M.O.), day/week high-low labels, REHs/RELs	1h or lower
Sessions	Asia, London, NY AM and NY PM high and low lines, optional rectangles, labels on unbroken levels	1h or lower
Premarket range	Range high and low, start and end vertical lines, quadrants, stddevs	5m or lower
NDOG / NWOG	New day and new week opening gap hi/lo lines and labels, optional combining of nearby gaps	1h or lower
RTHOG	Regular trading hours gap: settlement, open, CE, quadrants, stddevs, with a hint from 15 minutes before the open	15m or lower
EQH / EQL	Unswept equal highs and lows from the chart timeframe and from daily swings	1h or lower
Months	High, low and close of the last three months	1h or lower
Status table	Active session, macro and silver bullet names, object cap warning	1h or lower

Platform setup

The script has its own timezone, `tz`, and ignores the chart timezone setting. Set the platform timezone to New York for NQ and Madrid for DAX so the clock you read matches the times the script uses.

`useUS` picks the market: the Zone input forces US or EU, and Auto selects EU when the symbol description or ticker contains DAX, GERMANY or FDX. `tz` is then `America/New_York` or `Europe/Madrid`.

All time-of-day logic works on `tsec`, the open time of the bar in seconds since midnight in `tz`, computed with `hour(time, tz)`, `minute(time, tz)` and `second(time, tz)`. No `timestamp()` call is used, so daylight saving changes are handled by the timezone and there is no dependency on the exchange timezone. `dow` is the day of the week in `tz`, and `newDay` and `newWeek` come from its changes between bars.

Window	US (New York)	EU (Madrid)
Regular trading hours (market)	0930-1615	0900-1730
New day opening gap	1700-1800	2300-0000
Asia	2000-0000	2300-0600
London	0200-0500	0800-1300
NY AM	0700-1130	1330-1800
NY PM	1330-1600	1900-2200
Premarket (Auto)	0700-0900	0630-0830
Silver bullet	1000-1100, 1400-1500	1000-1100

Macros are ten fixed windows per market. The names and times sit in the arrays `mNames` and `mTimes`.

#	US name and window	EU name and window
1	NYpre 0740-0810	Dax7 0700-0710
2	NYAM.1 0850-0910	Dax8 0750-0810
3	NYAM.2 0950-1010	DaxOpen 0850-0910
4	NYAM.3 1050-1110	Dax10 0955-1010
5	NY.lunch 1150-1210	Dax11 1050-1110

#	US name and window	EU name and window
6	NYPM 1310-1340	DaxTapas 1245-1310
7	NYLast 1515-1530	Dax17 1650-1710
8	MOC 1545-1600	NYopen 1515-1540
9	LDN.1 0230-0300	Dax19 1850-1910
10	LDN.2 0403-0430	DaxLunch 1350-1410

The EU Asia window runs 2300-0600 Madrid time, and the EU "NY" windows are fixed Madrid clock times. US and EU switch daylight saving on different dates, so for about two weeks a year the New York open falls an hour away from those Madrid times.

Inputs

Inputs are grouped by module, and related controls share a row with `inline`. Colors, line styles and widths are configurable for every level type. Style strings are turned into Pine line styles by `getstyle()` and label sizes by `getsize()`.

Group	Inputs and effect
Time & Settings	Zone (Auto, US, EU); compute for historic data use (`visible`); offset for margin labels (`xoff` , bars between the last bar and the labels); show object cap warning (`showcap`)
Month Settings	Draw max/min/close of the last 3 months; colors for max, min and close; line style and width
Day Settings	Number of days (`ndays` , capped by the object budget); day and week vertical lines; day max/min colors, style and width; label unbroken day max/min and its size; REH/REL labels with a tolerance in daily ATR; equilibrium line; day open (midnight open) line
Session Settings	Draw sessions and how many to keep (`nses`); session rectangles and their transparency; no rectangles at or under a timeframe (default 5m); session max/min colors, style and width; label unbroken session max/min and its size; Asia, London and NY colors on one row

Group	Inputs and effect
Gaps and Ranges	Draw NDOG/NWOG/RTHOG and how many to keep (`ngaps`); separate NDOG, NWOG and RTHOG toggles; RTHOG quadrants and stddevs with their minimum sizes and count; gap rectangles; combine gaps with a tolerance in daily ATR; line style, width and color for NDOG, NWOG, RTHOG (old and current), quadrant and stddev lines; extend old RTHOG lines
Premarket Range	Draw premarket range; number of ranges that get quadrants and stddevs; Auto or Manual range with a session input (default 0700-0900 in `tz`); quadrants and stddevs with minimum sizes; stddev count; line color (red by default), width and the quadrant, CE and stddev styles; label size; tolerance for combining labels with RTHOG
Equal Highs / Lows	Draw EQH/EQL; also from the daily chart; pivot lengths for the chart and daily; tolerance in ATR; levels kept per side; colors (green EQH, red EQL), line style and width; label size
Macros and Events	Show the current session, macro and silver bullet names in the top-right table; optional background color during macros

Two inputs change how the script computes rather than how it looks. `visible` switches the processing window from the last bar to the rightmost visible bar, which recomputes when you scroll. `ndays` , `nses` and `ngaps` are upper bounds: the object budget can reduce them, and the table says so.

Data structures

Each concept is a user-defined type. An instance holds both the price and time data of the level and the handles of the chart objects drawn for it, so deleting the instance means deleting its objects first.

Type	Role	Key fields
`Sty`	Line style bundle	`sty` (style string), `wid` , `col`

Type	Role	Key fields
`ZoneInfo`	A recurring time window	`name`, `ss` and `es` (start and end in seconds of day), `sty`
`Zone`	One instance of a day, a session or a premarket range	`tstart`, `tend`, `op`, `hi`, `lo`, `hit` and `lot` (times of the extremes), `hidonet` and `lodonet` (time each extreme was taken), `extendt`, flags `isday`, `isweek`, `dead`, `drawmaxmin`, `drawrect`, `drawvlines`, `drawlbl`, week extremes `pwhi` and `pwlo`, REH/REL modes `hmode` and `lmode`, line, label and box handles, and the arrays `qlines`, `qlabels`, `sdlines`, `sdlabels` for premarket quadrants and stddevs
`Gap`	A new day or new week opening gap	`name`, `tstart`, `hi`, `lo`, `isw`, line, label and box handles; combining state `n` (gaps grouped), `nd` (NDOGs merged into a NWOG), `show`, and the combined levels `uhi`, `ulo`, `ut`
`RTHOG`	One regular trading hours gap	`tstart` and `cl` (settlement), `tend` and `op` (open), handles for the settlement, mid, open and quadrant lines and labels, a box, the arrays `po3lines` and `po3labels`, and `isold`
`Piv`	A swing high or low	`p` (price), `t` (time), `eq` (it became an equal level), `ln`, `lb`
`MonLn`	A month line and its label	`ln`, `lbl`

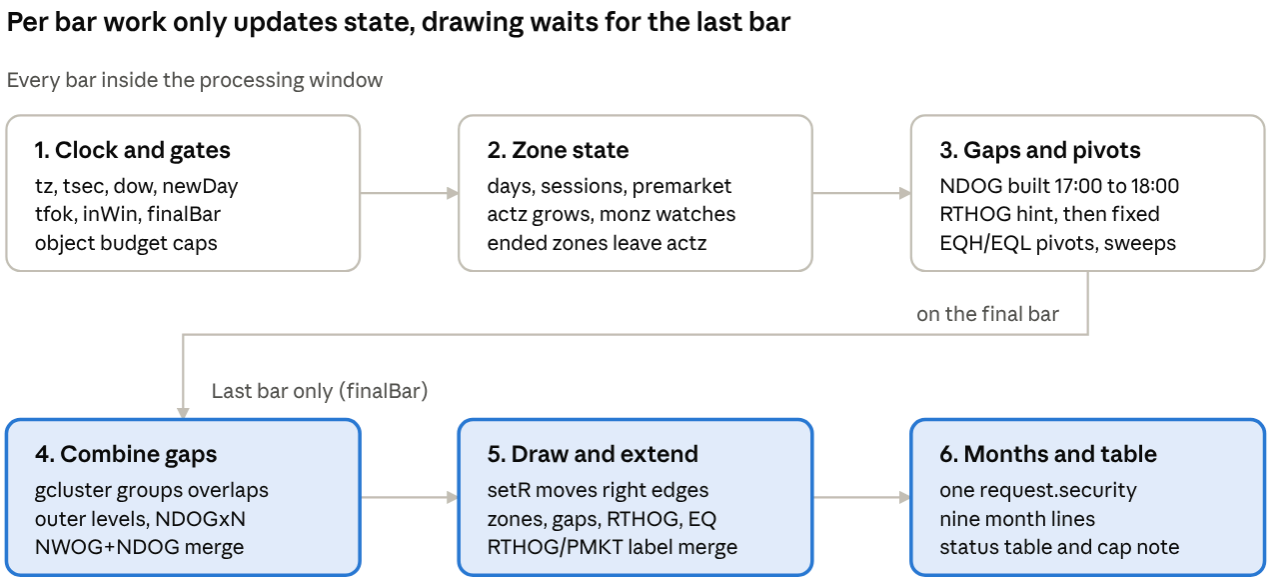
Global state is declared with `var` so it persists across bars.

Variable	Type	Content
`days`	`array<Zone>`	The last `ndaysEff` days, oldest first
`zones`	`array<Zone>`	Asia, London, NY AM and NY PM instances, up to `nsesEff`
`premarkets`	`array<Zone>`	Premarket range instances, up to `nsesEff`

Variable	Type	Content
`actz`	`array<Zone>`	Zones still growing: the only ones updated on every bar
`monz`	`array<Zone>`	Ended zones whose high or low has not been taken yet: only watched for sweeps
`dgaps`, `wgaps`	`array<Gap>`	Completed NDOGs and NWOGs, up to `ngapsEff` each
`ndog`	`Gap`	The gap under construction between the 17:00 close and the 18:00 open
`rthogs`	`array<RTHOG>`	Recent regular trading hours gaps; the last is current and the one before is drawn in the old color
`hpv`, `lpv`, `hpvD`, `lpvD`	`array<Piv>`	Candidate and equal highs and lows, for the chart timeframe and for daily swings
`mls`	`array<MonLn>`	Nine month lines (max, min, close for three months)
`zinfos`, `macros`, `sbs`	`array<ZoneInfo>`	Session, macro and silver bullet windows
`tbl`	`table`	Top-right status table

Execution pipeline

Pine runs the whole script once per historical bar and once per realtime tick, so the design keeps per-bar work small. Every bar inside the processing window only updates state. Objects are created and moved only when `finalBar` is true.



execution pipeline · per bar versus last bar

Processing window. `winEnd` is `timenow`, or the time of the rightmost visible bar when `visible` is on. `spanMs` is 1.5 times the number of days covered by the larger of `ndaysEff` and `ngapsEff`, plus a margin. `inWin` is true for bars between the two, and `finalBar` is true on the last of them (`barstate.islast`, or the first bar that closes past the visible edge). Bars outside the window cost almost nothing.

Gates. `tfok` (chart timeframe of 1h or lower) guards almost everything. `preOn` also requires 5m or lower and `rthOk` 15m or lower, so a module silently stops at coarser timeframes.

Initialisation. The two `barstate.isfirst` blocks build the `ZoneInfo` windows, the macro and silver bullet arrays and the status table.

Cost control. Only `actz` zones are updated per bar. A zone moves to `monz` when it ends and is dropped when its high and low are both taken or it falls out of the day budget. Combining gaps, label merging, month lines and the table are derived from state on the last bar, so they never accumulate drift.

Module logic

Days and weeks

On `newDay` a `Zone` with `isday` set is created. The previous day is closed by `endZone`, which stamps `tend` and moves the zone to `monz` so it is watched until its high and low are taken. The day before that gets `extendt`, which stops its equilibrium line. If the new day is the first of a week (`newWeek`), the running week extremes `pwhi` and `pwlo` restart;

otherwise they carry the maximum and minimum of the days of the week so far. A day's high label reads "week high" when it is at or above `pwhi`, and "day high" otherwise (same for lows). Beyond `ndaysEff` days the oldest zone is killed.

`markRel` implements REHs and RELs. It compares every pair of unbroken day highs (and lows) within a tolerance of `reltol` times the daily ATR. Within a cluster the highest high keeps its label and reads "day REHs" (mode 1), the others lose it (mode 2). Lows work the same way in reverse. The lines are never hidden.

Sessions

For every `ZoneInfo` in `zinfos`, `starts()` fires on the bar that contains the start instant and creates a zone. `ends()` fires on the bar that contains the end instant, or when the session was left (data gaps), and works for overnight windows because it runs on `tsec` modulo a day. While a zone is in `actz`, `upd()` extends its high and low and records the bar times of the extremes. A session level is labelled only until it is taken.

Premarket range

The range is a `Zone` from `premarket`, drawn with start and end vertical lines. Its window is `0700-0900` in `tz` by default, or the Auto times (07:00-09:00 for US, 06:30-08:30 for EU), or a manual session string. On the last bar the newest `pmkeep` ranges call `pmdraw`, which builds quadrants at 25%, 50% and 75% of the range when a quarter of it exceeds the minimum size, and stddev lines above and below the range. The stddev unit is the range divided by 8, 4, 2 or 1, taking the first that exceeds the minimum, and `pmpo3num` lines are drawn per side. Older ranges call `pmclear`.

NDOG and NWOG

The gap is built in `ndog`. At the bar before the gap window opens (`gapCloseBar`) or on the opening bar, the settlement price becomes one edge. A Friday or Sunday gap is a NWOG, any other is a NDOG. A fallback creates the NWOG when Sunday follows Friday with no bar in the gap window. On `gapEndBar`, `gend` takes the open as the other edge, orders `hi` and `lo`, names the gap with its date, draws it and pushes it to `dgaps` or `wgaps`, dropping the oldest beyond `ngapsEff`.

Combining gaps

When `combgaps` is on, the last bar runs `combineGaps` with a tolerance of `combtol` times the daily ATR. `gcluster` repeatedly merges gaps whose ranges overlap or are within the tolerance into groups, by relabelling a component id until nothing changes. The oldest gap of each group draws the outer levels (`uhi`, `ulo`, from `ut`) and counts the group in `n`, and the rest are hidden. Then every visible NWOG is compared with every visible NDOG:

if both of its edges are within the tolerance of the NWOG's edges, the NDOG is hidden and counted in the NWOG's `nd`. `gname` builds the text from that state: NDOGx3, NWOGx2 or NWOG+NDOG, always followed by the date of the gap that draws the level. `glabels` draws one label in the middle when high and low are within the tolerance, otherwise `.hi` and `.lo` labels.

RTHOG

The gap runs from the settlement of the last regular session bar (`mktClose`) to the next open. At the close a new `RTHOG` is created with `rstarts`, which draws the settlement line. From 15 minutes before the open (`rthStarting`) `rends` is called on the final bar with the current close as a hint, and on the open bar (`rthOpenBar`) with the real open, which fixes it. Each call deletes and rebuilds the CE, open, rectangle, quadrant and stddev lines, so the hint follows the price. Quadrants are placed at 25% and 75% of the gap, stddevs use the same 8, 4, 2, 1 rule as the premarket range. The current gap is drawn in the current color, the previous one in the old color, and older ones are reduced by `setOld` to short ticks around the close and open, optionally extended to the right. The gate `rthOk` requires 15m or lower, the last 5000 bars, and no Sunday bar.

Stddev label merging

`combinePo3` runs on the last bar when the premarket range and the RTHOG both exist. It renames each stddev label by its position in the sequence +1, -1, +2, -2 (`po3idx`) and, if a pair of lines is within `pmctol` times the daily ATR, writes one label such as "RTHOG +2 PMKT +3" on the RTHOG line and empties the other.

EQH and EQL

Pivots come from `ta.pivothigh` and `ta.pivotlow` on the chart timeframe and, with `doeqd`, from a daily `request.security` call that reads only the previous completed daily bar (`[1]` with `lookahead_on`), so nothing repaints. The tolerance is `eqtol` times the ATR of the same timeframe. `addPiv` merges a new pivot into an existing one within tolerance and marks it `eq`, keeping the more extreme price. `sweepPivs` removes a pivot when the price takes it: an unconfirmed candidate gets the tolerance as slack, a confirmed equal level none. `trimPivs` keeps `eqkeep` equal levels and at most 30 candidates per side. Only equal levels are drawn, by `drawEq`.

Months

One `request.security` call on `M` returns high, low and close of the previous three months, with the bar times. `drawMonth` creates each line once and then only updates it, extended to the right, with a label such as "max Sep".

Macros and table

Macro, session and silver bullet windows are tested with `inside(tsec)`. During a macro the chart background can be tinted, and the table shows the names of the active session, macro and silver bullet, plus a warning when the object cap forced limits.

Function reference

Methods are called on their object (`z.refresh()`); plain functions are called by name. Every function that would otherwise return nothing ends in `true`, because Pine requires a return value for the script to compile.

Drawing helpers

Function	Behavior
`rmLine`, `rmLabel`, `rmBox`	Delete the object if it is not `na`
`tright(n)`	Time of `n` bars to the right of the current bar, used as the right edge
`timestr()`	Current bar date as "d MMM" in `tz`
`mkLL(x1, x2, p, c, sty, w, txt, tc, sz)`	Creates a horizontal line and a left-aligned label at its right end, returns both
`setR(l, b, x, c)`	Moves the right end of a line and its label to `x` and recolors them
`tinyLbl(cur, show, x, y, txt, c, sz)`	Creates, moves or deletes a small label according to `show`, returns the label
`getstyle`, `getsize`	Convert input strings to `line.style_*` and `size.*` constants

Time windows

Function	Behavior
`parseSecs(s)`	Converts "HHMM" to seconds of day
`mkZoneInfo(name, ival, sty)`	Builds a `ZoneInfo` from "HHMM-HHMM", or `na` if the name or interval is empty
`inside(z, t)`	Whether second `t` is inside the window, handling overnight windows

Function	Behavior
`starts(z)`	The current bar contains the window start
`ends(z)`	The current bar contains the window end, or the window was left between two bars
`mkZone(name, zi)`	New `Zone` seeded from the current bar

Zones

Function	Behavior
`upd(z)`	Extends `hi`, `lo` and the times of the extremes with the current bar, no drawing
`refresh(z)`	Creates or updates every object of the zone from its state
`live(z)`	True when the zone still needs right-edge updates
`endZone(z)`	Stamps `tend` once, refreshes and moves the zone to `monz` if its levels are drawn
`kill(z)`	Marks the zone dead and deletes all of its objects
`runActive()`	Per-bar loop over `actz`: updates, ends or removes zones
`runMon()`	Per-bar loop over `monz`: records when each extreme is taken and removes finished zones
`markRel(arr, tol)`	Sets the REH/REL label mode of each day zone
`pmdraw(z)`, `pmclear(z)`	Build or delete the premarket quadrants and stddevs

Gaps

Function	Behavior
`gmake(g)`	Draws the lines, labels and optional rectangle of a gap
`gend(g, xe)`	Closes the gap with the open price, orders the edges, names and draws it

Function	Behavior
`gname(g)`	Label text: plain, NDOGxN, NWOGxN or NWOG+NDOG, with the date
`glabels(g, h, l, tol, x)`	Two labels with .hi and .lo, or one centered label when the levels are close
`gright(g, tol)`	Right-edge update when gaps are not combined
`gcomb(g, tol)`	Draws a gap with its combined levels, or deletes it when hidden
`gcluster(arr, tol)`	Groups gaps whose ranges overlap or lie within `tol`
`combineGaps(dg, wg, tol)`	Clusters both lists, hides NDOGs inside a NWOG, then draws all
`kill(g)`	Deletes the objects of a gap

RTHOG

Function	Behavior
`rstarts(g, c, t)`	Records the settlement and draws its line
`rends(g, opv)`	Sets the open (hint or final) and rebuilds the CE, open, box, quadrants and stddevs
`rquads(g)`, `rpo3(g)`	Quadrant and stddev lines
`killq`, `killpo3`, `kill`	Delete the quadrants, the stddevs or everything
`setOld(g)`	Replaces a gap by compact ticks
`right(g, c)`	Last-bar right-edge update with a color
`po3idx(k)`, `combinePo3(g, z, tol)`	Stddev label names and merging with the premarket range

Pivots and months

Function	Behavior
`addPiv(arr, p, t, isHigh, tol)`	Adds a pivot or promotes an existing one to an equal level

Function	Behavior
`sweepPivs(arr, isHigh, tol)`	Removes pivots taken by the current bar
`trimPivs(arr)`, `dropFirst(arr, wantEq)`	Enforce the limits on equal levels and candidates
`drawEq(arr, txt, c)`	Draws or updates the equal levels
`getm(m, t, p, c, txt)`	Creates or updates one month line and label
`drawMonth(idx, h, l, c, t, tc)`	Draws the three lines of one month

Object budget and performance

TradingView allows 500 lines, 500 labels and 500 boxes per script and silently deletes the oldest when they are exceeded. The script avoids that by computing, before anything is drawn, how many objects each enabled module can need.

Step	Rule
Caps	`nsesEff` is `nses` capped at 40, `ngapsEff` is `ngaps` capped at 15
Reserve	`lnReserve` and `lbReserve` add the worst case of months (9), gaps, RTHOG, sessions, premarket ranges and stddevs, EQH/EQL, plus 10 spare
Per day	`lnPerDay` and `lbPerDay` count the lines and labels one day needs given the enabled options
Days	`ndaysEff` is `ndays` limited by `(500 - reserve) / perDay` for lines and for labels, and at least 1
Report	When anything was reduced `capped` is true, the table shows a warning, and the first bar logs the old and new values

The design choices that keep it fast:

- Per bar, only `actz` is updated and `monz` is checked for sweeps. Both lists are short.
- No object is created or moved per bar except the RTHOG hint during the 15 minutes before the open.
- Right edges, combined gaps, premarket drawing, label merging, months and the table run only when `finalBar` is true.

- The processing window stops history work beyond roughly 1.5 times the number of days kept.
- Higher timeframe data comes from two `request.security` calls, daily for EQH/EQL and monthly for months, each with `lookahead_on` and a `[1]` or later offset so nothing repaints.
- RTHOG is limited to the last 5000 bars.

Limitations and extension points

Limitations

- The code has not been compiled or run in TradingView. Expect to fix small compile or runtime errors on first load.
- The platform timezone must match the market (New York for NQ, Madrid for DAX) for the clock to read as the script assumes. The DAX session windows are fixed Madrid clock times and drift against New York during the weeks when US and EU daylight saving differ.
- Modules stop at coarser timeframes: 1h for most, 15m for RTHOG, 5m for the premarket range.
- The budget is computed from the enabled options before drawing, so turning many options on reduces the number of days that can be kept.
- Combined gap levels are derived from the lists on each final bar. They are not stored, so scrolling in `visible` mode recomputes them.
- Equal highs and lows are detected only after the pivot is confirmed, so they appear `eqlen` bars late.
- An empty `for i = 0 to n - 1` loop runs backwards in Pine when `n` is 0, so every such loop needs an `n > 0` guard. This caused the blank chart when NDOGs or NWOGs were disabled.

Extension points

- A new session: add a `ZoneInfo`, put it in `zinfos` and add its inputs. It is then created, updated, ended and drawn by the existing loops.
- A new macro or silver bullet window: add its name and time to `mNames`, `mTimes` or `sbTimes`.
- A new gap kind: create the `Gap` where `ndog` is built, push it to its own list, and add that list to `combineGaps`, the final-bar loop and the object budget.
- A new level family: add a type, a `var` array and a budget term, update its state on every bar, and draw it only in the `finalBar` block.